

리스팅 vs 배열

	리스트(List)	딕셔너리(Dictionary)
기호	[] (대괄호)	{ } (중괄호)
핵심요소	Value 들만 나열	Key : Value (쌍으로 존재)
데이터접근	변수명[인덱스]	변수명[key]
특징	순서(인덱스)가 중요함	이름(Key)으로 값(Value)을 찾음 순서가 중요하지 않음

1. 생선 및 조류
2. 수산물 및 가공품
3. 삭제

1. 생성 및 접근

```
- 리스트
menu = ["커피", "에이드", "빵"]
print(menu[0])

- 딕셔너리
menu = {"americano": "coffee",
        "latte": "coffee",
        "baguette": "bread"}
print(menu["latte"])
```

2. 수정 및 추가

- 리스트
menu[2] = "과자"
menu.append("케이크")
- 딕셔너리
menu["baguette"] = "pain"
menu["espresso"] = "coffee"

3. 삭제

- 리스트
menu.remove("에이드")
- 딕셔너리
menu.pop("latte")

4. 특화된 문법(리스트)

- 슬라이싱
변수[시작인덱스 : 끝인덱스]

menu[0:2]

4. 특화된 문법(딕셔너리)

- 전체 조회
변수명.keys()
변수명.values()
변수명.items()

menu.keys()
menu.values()
menu.items()

```
menu = ["커피", "에이드", "빵", "과자", "케이크"]  
menu_price = [4000, 4500, 3500] (순서가 중요)
```

VW 디젤너리로 수정 VW

```
menu = {"커피": 4000, "에이드": 4500, "빵": 3500} (순서가  
중요)
```

```
# DB에서 가져온 추상화된 데이터 (컴퓨터 최적화)  
db_data = [0, 2, 1, 0]
```

```
# 이 숫자가 진짜 무엇인지 설명해주는 암호표  
translator = {0: "커피", 1: "에이드", 2: "뽕"}
```

-> 나중에 enum으로 디벨롭

리스트 : 미디어(사진, 동영상 등) 인코딩 과정에 사용

딕셔너리 : json 구조와 동일(key-value)

-> NoSQL 등에서 활용

```
order_json = {  
    "order_id": 20260615,  
    "items": [0, 1, 1, 2],  
    "total_price": 7500  
}
```